kind
Language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-630004.1
Haskell wiki: https://wiki.haskell.org/Kind

polykind

e.g. check the kind in the REPL
With StarIsType

```
ghci> :k Int
Int :: *
ghci> :k Maybe
Maybe :: * -> *
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: k -> *
```

Without StarIsType + explicit forall

```
ghci> :set -XNoStarIsType -fprint-explicit-foralls
ghci> :k Int
Int :: Type
ghci> :k Maybe
Maybe :: Type -> Type
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: forall {k}. k -> Type
```

Kind signature
usage 1: standalone kind signature

```
{-# LANGUAGE StandaloneKindSignatures -#}
{-# LANGUAGE TypeFamilies -#}

import Data.Kind

type D :: Type -> Type               -- data type
signature
data D a = MkD a

type T :: Type -> Type               -- type synonym
signature
type T a = Maybe a

type F :: Type -> Type               -- type family
signature
type family F a where
  F Int = String

type C :: Type -> Constraint         -- type class
```

```
signature
class C a where
```

usage 2: inline kind annotation

```
data D (a :: Type) = MkD a           -- data type
variable annotations

f :: (a :: Type) -> (a :: Type)      -- function type
variable annotations

class MyClass (a :: Type) where      -- class type
variable annotations
```

```
{-# LANGUAGE StanadloneKindSignature #-}
{-# LANGUAGE NoStarIsType #-}
import Data.Kind

-- case: basic
type A :: *
type A :: Type
type A :: Type -> Type
type A :: Type -> Type -> Type         -- binary
constructor
type A :: (Type -> Type) -> Type       -- unary of
unary. NOT the same as binary
```

```
-- e.g.
Int :: *
Int :: Type
() :: Type

-- case: nary
Either :: Type -> Type -> Type
Either Int :: Type -> Type
Either Int Int :: Type
Either Maybe Int                   -- ERROR. Maybe is not
nullary

-- case: nested constructors
type A :: (Type -> Type) -> Type
data A m = MkA (m Int)
MkA [1] :: A []
MkA (Just 1) :: A Maybe
MkA 1                              -- ERROR
```

```
-- case: forall, polykinds
--
{-# LANGUAGE ExplicitForall #-}
{-# LANGUAGE PolyKinds #-}
```

```
type A :: a -> Type                              --
polykind
type A :: forall a. a -> Type                    -- same
explicit
type A :: forall a b. a -> b -> Type              --
multiple
```

```
{-# LANGUAGE ExplicitForall #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE DataKinds #-}

-- case: Type only
type T :: a -> Type
type T :: forall a. a -> Type
data T a = MkT

MkT :: T Int                --
MkT :: T Maybe              -- partially applied kind
Maybe :: Type -> Type
MkT :: T 'True              -- custom kind 'True :: Bool
```

```
-- case: data kinds, type literals
{-# LANGUAGE DataKinds #-}
import GHC.TypeLits
type A :: Nat -> Symbol -> Type
type A :: Bool -> Type
```

```
e.g.
{-# LANGUAGE DataKinds #-}
import GHC.TypeLits
data Unit = F | C

type A :: Unit -> Nat -> Bool -> Type
data A a b c = MkA

MkA :: A 'F 1 'False
MkA :: A 'C 2 'True
```

type synonym arity vs kind
Standalone kind signatures: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/poly_kinds.html#standalone-kind-signatures-and-polymorphic-recursion
Type families: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html#type-family-declarations

type synonym arity = **type parameters** of a type synonym can NOT be partially applied
e.g. type MyType m a = [m a]
parameters = m a

arity = 2
type kind (signature) = **type variables** of a type constructor

CAN be partially applied

This is needed to differentiate a type synonym of a **normal kind** vs **higher ordered kind**

type A = Maybe	synonym as higher-ordered kind
type B a = Maybe a	synonym as normal kind
f :: Functor A => ...	OK
f :: Functor B => ...	ERROR "The type synonym ‘B’ should have 1 argument"

e.g. no partial application of arity

```
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Proxy
import Data.Kind

type Container :: Type -> Type        -- kind Type ->
Type
data Container a = Container a

type C :: Type -> Type                -- same kind as
Container
type C a = Container a                -- arity=1

f :: Proxy Container                  -- OK
f :: Proxy C                          -- ERROR "The type
synonym M should have 1 argument"
```

Example: same kind but different arity
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/poly_kinds.html#standalone-kind-signatures-and-polymorphic-recursion
Type families in haskell: https://serokell.io/blog/type-families-haskell#type-constructor-arity

All the types below have kind
forall a. a -> Type

```
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}

import Data.Proxy
import Data.Kind

-- arity T1 = 1
type T1 :: forall a. a -> Type
type T1 = Proxy                               -- params=a
 arity=1
type T1 = Maybe                               --
ERROR "expected kind 'a -> *' but has kind '* -> *'"

t1 :: T1 @Type Int                            -- a=Type
t1 = Proxy
```

```
t1b :: T1 @(Type -> Type) Maybe             -- a=Type-
>Type
t1b = Proxy
t1c :: T1 @(Type -> Type -> Type) Either    -- a=Type-
>Type->Type
t1c = Proxy

-- arity T2 = 2
type T2 :: forall a. a -> Type
type T2 b = Proxy b                         --
params=a,b  arity=2

t2 :: T2 @Type Int                          -- a=Type
b=Int
t2 = Proxy
t2b :: T2 @(Type -> Type) Maybe             -- a=Type-
>Type b = Maybe
t2b = Proxy
t2c :: T2 @(Type -> Type -> Type) Either    -- a=Type-
>Type->Type b=Either
t2c = Proxy
```

Type synonym families

```
newtype Apply1 m = Apply1 (m Int)
newtype Apply2 m = Apply2 (m Int Int)

-- arity F0 = 0
type F0 :: forall a. a -> Type
type family F0 :: forall b. b -> Type where      --
params=NA   arity=0
  F0 = Proxy                                     --
saturated=F0
  F0 = Maybe                                     --
ERROR "Expected kind 'forall a. a -> *' but Maybe has
kind '* -> *'"

-- arity F1 = 1
type F1 :: forall a. a -> Type
type family F1 :: b -> Type where         -- params=a
  arity=1
  F1 @Type = Maybe                        -- a=Type
  F1 @(Type -> Type) = Apply1             -- a=(Type ->
Type)
  F1 @(Type -> Type -> Type) = Apply2

-- arity F2 = 2
type F2 :: forall a. a -> Type            -- params=a,b
 arity=2
type family F2 b :: Type where
  F2 @Type Int = Int                      --
saturated=F2 @Type Int
  F2 @(Type -> Type) Maybe = Bool         --
 saturated=F2 @(Type -> Type) Maybe
```

Inferred kind signatures
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/poly_kinds.html#overview-of-kind-polymorphism

```
---- Value without right hand side
--
{-# LANGUAGE PolyKinds -#}
f :: Proxy a                          -- forall {k}
(a :: k)    i.e. polykind
f :: Proxy (a :: k)                   -- a :: Type
          defaults to k = Type
f :: Proxy (a :: k -> k)              -- a :: Type
       defaults to (k->k) = Type

-- forall
f :: forall {k} (a :: k). Proxy a     -- polykind
f :: forall a. Proxy a                -- same i.e.
polykind
f :: forall k (a :: k). Proxy a       -- a :: Type
       i.e. k=Type by default
f :: forall (a :: Bool). Proxy a      -- data kind
```

```
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE DataKinds #-}
import Data.Kind

-- case: polykind
f :: Proxy a                      -- polykind
f :: forall a. Proxy a
f :: forall {k} (a :: k). Proxy a
f = Proxy
f @Int                            -- Proxy (Int :: Type)
f @Maybe                          -- Proxy (Maybe :: Type
-> Type)     unary
f @'True                          -- Proxy ('True ::
Bool)             data kind

-- case: Type only
f :: Proxy (a :: Type)
f :: Proxy (a :: k)                   -- same. Defaults
to Type
f :: Proxy (a :: k -> k)              -- same
f :: forall k (a :: k). Proxy a
f @Int         -- Proxy (Int :: Type)
f @Maybe       -- ERROR "expected kind Type"
f @'True       -- ERROR "expected kind Type"

-- case: data kind
f :: Proxy (a :: Bool)
f @Int         -- ERROR "expected kind Bool"
f @Maybe       -- ERROR "expected kind Bool"
f @'True       -- Proxy ('True :: Bool)
```

```
---- Value with right hand side
--
type family F a                          -- type F :: Type
-> Type
type family F (a :: Type)                -- same
type family F (a :: k)                   -- type F ::
forall {k} (a :: k). a -> Type      i.e. polykind
type family F (a :: Bool)                -- type F ::(a ::
Bool) -> type                   i.e. data kind  a
~ 'True, 'False :: Bool
type family F a :: k                     -- type F ::
forall {a} {k}. a -> k              i.e. polykind in
both?
type family F (a :: k) :: k              -- type F ::
forall {k} (a::k). a -> a           i.e. polykind in
both?
type family F (a :: k) :: l              -- type F ::
forall {k} {l}. k -> l              i.e. polykind in
both
```

==========
======

GHC extension

GHC extension: Kind signature
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/kind_signatures.html#extension-KindSignatures

Kind signature
Annotating the **kind** of a type variable.
i.e. whether a type variable is nullary type, unary type, binary type, etc

```
e.g.
{-# LANGUAGE KindSignatures #-}
import Data.Kind

data A a = A a                              -- a=nullary
e.g. a=Bool
data A (a:: Type) = A a                     -- a=nullary
e.g. a=Maybe x, Either Int x
data A (a:: Type -> Type) b = A (a b)       -- a=unary
b=nullary  e.g. a=Either x y
```

```
-- Basics
{-# LANGUAGE KindSignatures #-}
import Data.Kind

data A (a:: Type) = A a                       -- nullary
data A a = A a                                -- same
data A (a:: Type -> Type) b = A (a b)         -- unary
data A a b = A (a b)                          -- same
```

```
e.g.
x = A (Just 1)
```

```
data A (a:: Type -> Type) b = A a b          -- ERROR. a
is defined as unary but used as nullary a
data A (a:: Type) b = A (a b)                -- ERROR. a
is defined as nullary but used as unary (a b)
data A (a:: Type -> Type -> Type) b = A (a b c)
  -- binary
data A (a:: Type -> Type) b = A (a b c)      -- ERROR. a
is defined as unary but used as binary (a b c)

-- usage: declarations
data A (a:: Type -> Type) = A (a Int)           -- in
data
type A (a:: Type -> Type) = f Int               -- in
type
class A (a:: Type -> Type) b where              -- in
class
f :: (a :: Type) -> (b :: Type) -> a            -- in
function
-- existential quantification
{-# LANGUAGE ExistentialQuantification #-}
data A = forall (a:: Type). Eq a => A a

-- usage: in explicit foralls
{-# LANGUAGE ExplicitForAll #-}
f :: forall (a :: Type). Proxy a
f :: forall {k} (a :: k). Proxy a               --
polykind
```

```
f :: forall {k} (a :: k). Proxy a
f = Proxy
f @Int            -- OK on Int :: Type
f @Maybe          -- OK on partially applied Maybe ::
Type -> Type
f @'True          -- OK on data kind 'True :: Bool
```

GHC extension: Kind polymorphism
Ref https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/poly_kinds.html

Kind polymorphism
Polymophism in the kind
i.e. **variable** in the kind signature

```
-- monomorphic kind
type A :: Type
data A = A

-- monomorphic kind (polymorphic type)
type A :: Type -> Type
data A a = A
```

```
  A :: A Int                                  Int :: Type
  A :: A [Int]
  A :: A []                                   ERROR since [] :: Type ->
TYpe

  -- polymorphic kind
  type A :: a -> Type
  type A :: forall a. a -> Type
  A :: A (Int :: Type)                        a :: Type
  A :: A (Maybe :: Type -> Type)              a :: Type -> Type
  A :: A []
  A :: A ('True :: Bool)                      a :: Bool     i.e. data kind
```

e.g. type + polymorphic kind
without polymorphism

```
newtype MyType f a = MkMyType (f a) deriving (Show)
         -- type MyType :: (Type -> Type) -> Type ->
Type

        -- i.e. f=unary, a=nullary
-- e.g.
newtype MyCont c = MkMyCont (c Int)
MkMyType (Just 1) :: MyType Maybe Int
  -- OK f=Maybe=unary a=Int=nullary
MkMyType (MkMyCont [1]) :: MyType MyCont []
 -- ERROR. f=MyCont=unary a=[]=unary
```

with polymorphism

```
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Kind

type MyType :: forall a. (a -> Type) -> a -> Type
newtype MyType f a = MkMyType (f a) deriving (Show)



-- e.g.
newtype MyCont c = MkMyCont (c Int)
MkMyType (Just 1) :: MyType Maybe Int              -- OK
f=Maybe=unary a=Int=nullary
MkMyType (MkMyCont [1]) :: MyType MyCont []        -- OK
f=MyCont=unary a=[]=unary
```

e.g. type class with polymorphic kind
without polymorphism

```
class MyLabel u where                  -- type :: Type ->
Constraint
  getLabel :: Proxy u -> String

-- e.g.
instance MyLabel Int where             -- OK. u = Int =
```

```
nullary
  getLabel _ = "mylabel: Int"

instance MyLabel Maybe where          -- ERROR.
u=Maybe=unary
```

with polymorphism

```
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Kind

type MyLabel :: forall a. a -> Constraint
class MyLabel u where
  getLabel :: Proxy u -> String

-- e.g.
instance MyLabel Int where            -- u = Int =
nullary
  getLabel _ = "mylabel: Int"
instance MyLabel Maybe where          -- u = Maybe =
unary
  getLabel _ = "mylabel: Maybe"
```

Syntax

```
-- Kind polymorphism in kind signature
type A :: Type -> Type                 -- Normal
type A :: a -> Type
type A :: forall a. a -> Type          -- Explicit


-- Kind polymorphism in type signature
f :: forall k (a :: k). ....
```

GHC Extension: DataKind (data type promotion)
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/data_kinds.html

Data type promotion
By default, there is only one kind for concrete type i.e. the **Type** kind
Data types are **promoted** to kinds

```
-- Data as type
x :: Bool
x = True                    term=True , type=Bool
x :: Maybe Bool
x = Just False              term=Just False, type=Maybe Bool
type X :: Type
type X = Bool               type= Bool of kind=Type

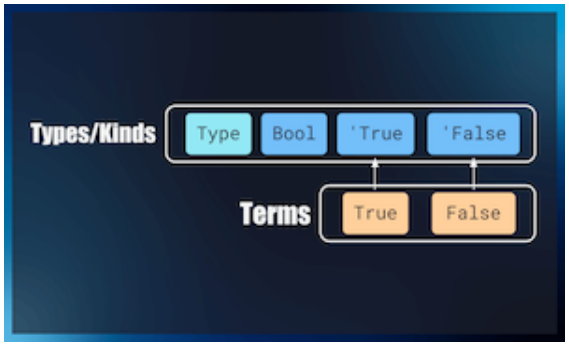
-- Data as kind
```

```
type X :: Bool
type X = 'True                  type='True, kind=Bool
type X :: Maybe Bool
type X = 'Just 'False           type='Just 'False, kind=Maybe Bool
```

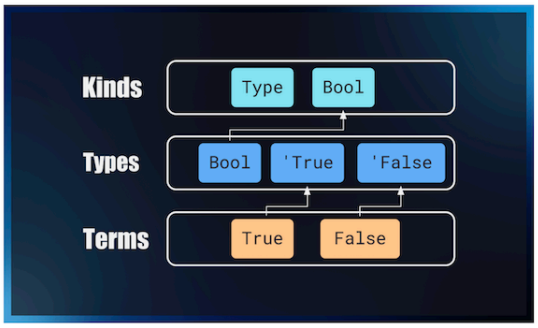
kind A vs type A
e.g. kind Bool vs type Bool
Data kinds are not what you think: https://serokell.io/blog/datakinds-are-not-what-you-think
There is NO distinction between type and kind
In particular, a **type parameter** can be used both as type and kind in a signature

```
f :: (a :: k) -> k          k used as kind in (a :: k)
                            k used as type in (-> k)
```

Correct



Wrong



e.g. same Bool

```
True :: Bool        Bool used as type
'True :: Bool       same Bool used as kind
```

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

import Data.Kind (Type)
import GHC.Types (Constraint)

type HasSing :: forall {k}. k -> Constraint
class HasSing (a :: k) where
    -- k as kind
```

```
  type Sing a :: Type
  sing :: Sing a
  demote :: Sing a -> k
    -- k as Type

withSing :: forall a r. (HasSing a) => (Sing a -> r) ->
r
withSing f = f (sing @a)
```

e.g. With Bool

```
data SBool (a :: Bool) where
  STrue :: SBool 'True
  SFalse :: SBool 'False

instance HasSing ('True :: Bool) where
    -- k=Bool as kind
  type Sing 'True = SBool 'True
  sing = STrue
  demote _ = True
    -- k=Bool as Type

instance HasSing 'False where
  type Sing 'False = SBool 'False
  sing = SFalse
  demote _ = False
```

usage: data unit kinds
e.g. temperature units

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE KindSignatures #-}

data TUnit = C | F deriving (Show)

-- e.g. TUnit as type
c :: TUnit
c = C                          -- value=C type=TUnit
c = F                          -- value=F type=TUnit

-- e.g. TUnit as kind
newtype Temp (u :: TUnit) = MkTemp Double
freezingC :: Temp 'C           -- type='C kind=TUnit
freezingC = MkTemp @'C 0
freezingF :: Temp 'F           -- type='F kind=TUnit
freezingF = MkTemp @'F 32
```

usage: metadata kinds
e.g. Person info as metadata

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE ExistentialQuantification #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}

import Data.Foldable
import Data.Kind
import Data.Proxy
import GHC.TypeLits

-- step: define kind PersonMeta for storing metadata
type PersonMeta :: Type

data PersonMeta = MkPersonMeta !Nat !Status !Color !
Symbol !Bool
data Color = Black | White
data Status = Approved | Denied

-- step: annotated with PersonMeta kind
type PersonRep :: PersonMeta -> Type
data PersonRep (a :: PersonMeta) = MkPersonRep
  { pid :: !Int,
    status :: !Status,
    color :: !Color,
    fullName :: !String,
    isAwesome :: !Bool
  }

-- step: reading the metadata
pidMeta :: forall (a :: Nat) (b :: Status) (c :: Color)
(d :: Symbol) (e :: Bool). KnownNat a => PersonRep
('MkPersonMeta (a :: Nat) b c d e) -> Int
pidMeta _ = fromInteger $ natVal (Proxy :: Proxy a)

fullNameMeta :: forall (a :: Nat) (b :: Status) (c ::
Color) (d :: Symbol) (e :: Bool). KnownSymbol d =>
PersonRep ('MkPersonMeta a b c d e) -> String
fullNameMeta _ = symbolVal (Proxy :: Proxy d)

-- e.g.
person1 :: PersonRep ('MkPersonMeta 101 'Denied 'Black
"Yui Nishizawa" 'False)
person1 =
  MkPersonRep
    { pid = 101,
      status = Denied,
      color = Black,
      fullName = "Yui Nishizawa",
      isAwesome = False
    }
person2 :: PersonRep ('MkPersonMeta 0 'Approved 'White
"Alyosha Karamzov" 'True)
person2 =
```

```
  MkPersonRep
    { pid = 0,
      status = Approved,
      color = White,
      fullName = "Alyosha Karamzov",
      isAwesome = True
    }

-- e.g. read metadata
pidMeta person1                -- 101
pidMeta person2                -- 0
fullNameMeta person1           -- "Yui Nishizawa"
fullNameMeta person2           -- "Alyosha Karamazov"
```

usage: metadata kinds in Generics
e.g.

```
-- GHC.Generics
data Meta = MetaData Symbol Symbol Symbol Bool
          | MetaCons Symbol FixityI Bool
          | MetaSel  (Maybe Symbol)
                     SourceUnpackedness SourceStrictness
DecidedStrictness

type BoolMeta :: Meta
type BoolMeta = 'MetaData "Bool" "GHC.Types" "ghc-prim"
'False

type BoolType :: M1
type BoolType x =
  M1 D ('MetaData "Bool" "GHC.Types" "ghc-prim" 'False)
(
    M1 C ('MetaCons "False" 'PrefixI 'False) U1
    :+:
    M1 C ('MetaCons "True" 'PrefixI 'False) U1
  ) x
```

```
---- Common kinds
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
data Alpha = A | B | C

-- Bool kind
type B0 :: Bool      -- kind Bool
type B0 = 'True      -- type 'True
type B1 :: Bool
type B1 = 'False

-- Tuple kind
type X :: (Bool, Nat)
type X = '( 'True, 1)
```

```
type X = '('True, 1)          -- GOTCHA ERROR. Need
space between ( and '
type X = (True, 1)            -- GOTCHA ERROR.
Interpreted as type constructor (,) : Type, not kind
constructor '(,) : (a,b)
type X :: (Bool, Alpha)       -- Custom type
type X = '( 'True, 'A)
type X = '('True, 'A)         -- GOTCHA ERROR. Need
space between ( and '

-- List kind
type L0 :: [Nat]         -- kind [Nat] i.e. list of
numbers
type L0 = '[]            -- type '[] i.e. empty list
type L0 = []             -- ERROR. Interpreted as type
constructor [] : Type, not kind constructor '[] : [Nat]
type L0 = '[ 1]
type L0 = '[ 1,2]
type L0 = '[ 1,2]
type L3 :: [Symbol]
type L3 = '[ "abc","def"]
type L4 :: [Alpha]       -- user defined type
type L4 = '[ 'A, 'B]
type L4 = '['A, 'B]      -- GOTCHA ERROR. Need space
between [ and '
```

GHC Extension: Type level literals
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_literals.html#type-level-literals
GHC.TypeLits: https://hackage.haskell.org/package/base/docs/GHC-TypeLits.html

Type level literals
Numbers and strings as **kinds**

Numbers as kind

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Proxy
import GHC.TypeLits

-- literal as value
t0 :: Int         -- type Int
t0 = 0            -- value 0
t1 :: Int
t1 = 1

-- literal as type
type T0 :: Nat     -- kind Nat
type T0 = 0        -- type 0
type T1 :: Nat
type T1 = 1
```

```
type T0' :: Int     -- NO
type T0' = 0        -- ERROR error="Expected kind Int
but '0' has kind Nat"

-- e.g. get value
natVal :: KnownNat a -> p a -> Integer
natVal (Proxy :: Proxy T0)        -- 0
natVal (Proxy :: Proxy T1)        -- 1
```

Strings as kind

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Proxy
import GHC.TypeLits

-- literal as value
s0 :: String            -- type String
s0 = ""                 -- value ""
s1 :: String
s1 = "ABC"

-- literal as type
type S0 :: Symbol        -- kind Symbol
type S0 = ""             -- type ""
type S1 :: Symbol
type S1 = "ABC"

type S0' :: String
type S0' = ""           -- ERROR "Expected kind
'String', but '""' has kind 'Symbol'"

-- e.g. retrieve value
symbolVal :: KnownSymbol a => p a -> String
symbolVal (Proxy :: Proxy S0)         -- ""
symbolVal (Proxy :: Proxy S1)         -- "ABC"
```

usage: tag with numeric literal (Nat kind)
Fixed point integer

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Kind
import GHC.TypeLits (KnownNat, Nat, natVal)

type FixedPoint :: Nat -> Type
  -- Nat = Int as kind
newtype FixedPoint f = MkFixedPoint Integer

getValue :: (KnownNat f, Fractional b) => FixedPoint f
-> b
getValue p@(MkFixedPoint v) = fromInteger v /
```

```
fromInteger (natVal p)

instance KnownNat f => Show (Fixed f) where show =
show . (getValue)

print (MkFixedPoint 1234 :: FixedPoint 1)            --
1234.0
print (MkFixedPoint 1234 :: FixedPoint 10)           --
123.4
print (MkFixedPoint 1234 :: FixedPoint 100)          --
12.34
```

usage: tag with string literal (Symbol kind)
String type with prefix and prefix

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import Data.Kind
import Data.Proxy
import GHC.TypeLits (Symbol, symbolVal, KnownSymbol)

type SurroundedString :: Symbol -> Symbol -> Type
           -- Symbol = String as kind
newtype SurroundedString p s = MkSurroundedString String


instance (KnownSymbol p, KnownSymbol s) => Show
(SurroundedString p s) where
  show (MkSurroundedString v) = symbolVal (Proxy @p) ++
v ++ symbolVal (Proxy @s)

print $ MkSurroundedString "hello" :: SurroundedString
"[" "]"     -- "(hello)"
print $ MkSurroundedString "hello" :: SurroundedString
"[" "]"     -- "[hello]"
```

usage: use type as term i.e. reflection
e.g. map tag type to a corresponding String
2 steps:
type -> Symbol with type family
Symbol -> String with symbolVal

```
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
import GHC.TypeLits

data Answer = YES | NO

-- step 1: type -> Symbol
type ToSymbol :: forall a. a -> Symbol
```

```
type family ToSymbol a
type instance ToSymbol 'YES = "YES"
type instance ToSymbol 'NO = "NO"

-- step 2: type -> String
getAnswer :: forall a. (KnownSymbol (ToSymbol a)) =>
Proxy (a :: Answer) -> String
getAnswer _ = symbolVal (Proxy @(ToSymbol a))

-- e.g.
getAnswer (Proxy @'YES) = "YES"
getAnswer (Proxy @'NO) = "NO    "
```

GOTCHA: error converting type literal to value (e.g. symbolVal, natVal) "No instance for (KnownSymbol a)" or "No instance for (KnownNat a)"

Problem: restricting to Symbol isn't enough
Use KnownSymbol class instead of kind Symbol.

```
-- BAD: not enough to have Symbol
toString :: forall a. Proxy (a :: Symbol) -> String
          -- BAD: restricting to Symbol isn't enough
toString _ = symbolVal (Proxy @a)
          -- error "No instance for (KnownSymbol a)
arising from a use of 'symbolVal'"

-- GOOD: specify KnownSymbol instead
toString :: forall a. (KnownSymbol a) => Proxy a ->
String       -- GOOD: use KnownSymbol instead of Symbol
toString _ = symbolVal (Proxy @a)
```

==========
======

OLD

Int no polymorphism monomorphic
type Int of monomorphic kind Type

forall (a :: Type) **type polymorphism** polymorphic
type a of monomorphic kind=Type
a = Int :: Type
a = [Int] :: Type

forall (a :: Type -> Type) type polymorphism polymorphic
type a of monomorphic kind=Type->Type
a = Maybe :: Type -> Type
a = [] :: Type -> Type

forall k (a :: k) **kind polymorphsim** polymorphic

```
type a of polymorphic kind k
    a = Int :: Type                  kind Type
    a = Maybe :: Type -> Type        kind Type -> Type
    a = 'True :: Bool                promoted kind Bool
    a = 1 :: Nat                     literal kind Nat
```